AI Product Management Final Project Deliverables

Cappy Lassen, Aml Babikir, Julia Kazak-Nersesava, Karthikraja Raju, and Zakia Aliahmad

April 13, 2026

Hi! I'm your Deliverables Template 👋

Please duplicate me and customize your copy for your final presentation.

Individual slides have been provided for your groups, starting on slide 5. Each section aims to assist you in gathering and organizing the necessary information needed to submit your deliverables in the LMS to qualify for certification. Remember that every group member must submit a copy.

For additional context and FAQs, see the 🔗 **Project Brief.**

Pro Tip: This is just a base. Feel free to get creative or use AI presentation tools like Gamma, Canva, etc. to make it your own!

To do so, feed the information from slides 5 and beyond into your tool of choice and prompt a new presentation output.

Your Project Deliverables Should Include:

- **System Message:** Share the specific System Prompt you developed, highlighting the role, task, constraints, output, and the specific few-shot example you embedded to ensure Juno thinks like an Associate PM.
- **AI Strategy One-Pager Summary:** Summarize your strategic foundation and AI opportunity you identified, and include a link to your full Strategy One-Pager artifact.
- **RAG Architecture & Data Strategy:** Detail how Juno accesses information. Explain your data retrieval logic and the specific sources Juno uses to minimize hallucination.
- **UX Design:** Show Juno in action where the work happens. Provide 1-2 high-fidelity screenshots of Juno's interface.
- **Agentic Workflow (AW Spec):** Present the logic behind Juno's execution. Detail the core agentic pattern used to handle complex tasks and include a link to the full AW Spec artifact.
- **Evaluation Plan:** Summarize your methodology for testing Juno's outputs, the specific definition of "good" you established, and how you plan to mitigate risks.
- **Insights from the Build Process:** Reflect on the journey. Identify your group's biggest technical or strategic friction points, your key "aha" moment, and what you learned during the build process.

Final Project Deliverables

[Start building your presentation below!]

Core Instructions: System Message

Complete

System Message

Paste the final version of your system prompt you developed in Module 1 for Juno. You do not need to add the Format., unless you changed it.

Role: You are Juno, a sharp Associate AI PM Copilot at RocketShip. You specialize in risk-and-value prioritization and evidence-based product discovery. You are focused on weighing system reliability first, then revenue exposure, then UX polish..

Task: Your task is to analyze this snapshot of artifacts into a structured Opportunity Brief. Your goal is to identify the single most important problem to address next, rank the top opportunities, and explain why.

Constraints: Follow these rules strictly to ensure the Opportunity Brief is reliable:

- Never invent metrics or quotes, or user sentiment.
- Attribute every insight to its source using the provided source ID.
- If evidence for a claim is weak, conflicting, or based on a single source, say so explicitly, and give it a confidence score.
- Ensure confidence in the priority weight is explicit: Assign High, Medium, or Low based on number of sources, consistency across artifacts, and whether the claim is backed by direct evidence versus opinion.
- Combine duplicate signals that point to the same root cause.
- Separate symptoms from underlying problems whenever possible.
- Prioritize based on business impact, customer harm, and operational risk, not just how loudly a stakeholder asks.

Few-Shot Example

Highlight the specific example you provided in the prompt.

Below is an example of the quality I expect. Use this as a reference for tone, density, and evidence-linking:

- Example Problem: Enterprise deal is blocked due to missing SAML/SSO support, creating immediate revenue risk.
- Example Evidence: Prospect explicitly states they will not sign without SSO support. Deal deadline is tied to Oct 1st, indicating time-sensitive revenue impact
- Example Risk: High-confidence revenue blocker with time constraint; failure to deliver SSO will result in lost deal.

AI Strategy One-Pager

Complete

Link to Artifact from Module 2: ***AI Product Management M2: Three-Layer Model & AI Strategy***

Completed Sections

- Problem and Workflow: Product decisions at RocketShip are inconsistent and reactive, and the team prioritizes work based on the loudest requests or most recent issues. Juno prevents "Signal-Driven Bias", specifically making roadmap decisions based on scattered or anecdotal inputs instead of basing it on structured, evidence-based insights
- Target Metrics: **User KPI** - Reduction in time for intake to prioritization decision from 3 days to 2 hours. **Business KPI** - 20–30% faster sales cycle for deals involving product dependencies. For ~100 deals/year at a $50K average deal size with a 10% conversion rate, this translates to ~$500K in incremental revenue driven by faster, unblocked deal flow.
- Autonomy Level: **Choose Copilot** - Synthesizes inputs and recommends priorities, but a human PM makes the final call because prioritization is a high-impact decision. **Avoid Agent** - Juno shouldn't have full autonomy to commit roadmap changes or respond to Sales without human review. An incorrect decision such as greenlighting the wrong feature can have direct revenue and credibility impact.
- Data & Model Approach: We will **use RAG** to ground the model in RocketShip's relevant product documentation so that Juno's outputs are tied to current business context with traceable evidence instead of general assumptions. We are intentionally **avoiding** a **generic LLM** because it would rely too heavily on pattern matching and broad product knowledge without knowing RocketShip's actual customers, blockers, or internal decisions.

RAG Architecture & Data Strategy

Complete

Data Sources

List the specific sources Juno retrieves from in order to use RAG. Hint: Use the Data Requirements section of your AI PRD snippet from Module 3!

1. Slack: Product, Sales, and Support channels containing escalations, customer feedback, and deal blockers
2. Jira: Tickets, bugs, and feature requests with status, priority, and linked work
3. Zendesk (Support Tickets): Customer issues, tagged by category, severity, and volume
4. Confluence: Product specs, past decisions, and documented requirements

Data Strategy

Describe how Juno finds the right information to minimize hallucinations. Hint: Use the Model Requirements and AI User Experience sections of your AI PRD snippet from Module 3!

Model Requirements

- RAG enables Juno to retrieve the most relevant, up-to-date signals across Slack, Jira, and Zendesk at scale, using metadata (e.g., severity, recency, source).
- Long context allows Juno to synthesize those retrieved signals into a coherent view by identifying patterns, summarizing themes, and forming clear recommendations.
- This combination ensures Juno is both precise (grounded in the right data) and holistic (able to connect signals across sources), which is critical for accurate prioritization.

AI User Experience

- **Verification**: Only provide sources, when there are traceable sources indexed and referenced. Include links or reference IDs like ticket numbers in the source output.
- **Fail Safe**: If no sources *are retrieved, then inform the user that no sources were found in the output. Do not hallucinate or suggest false sources.*

UX Design & Interface

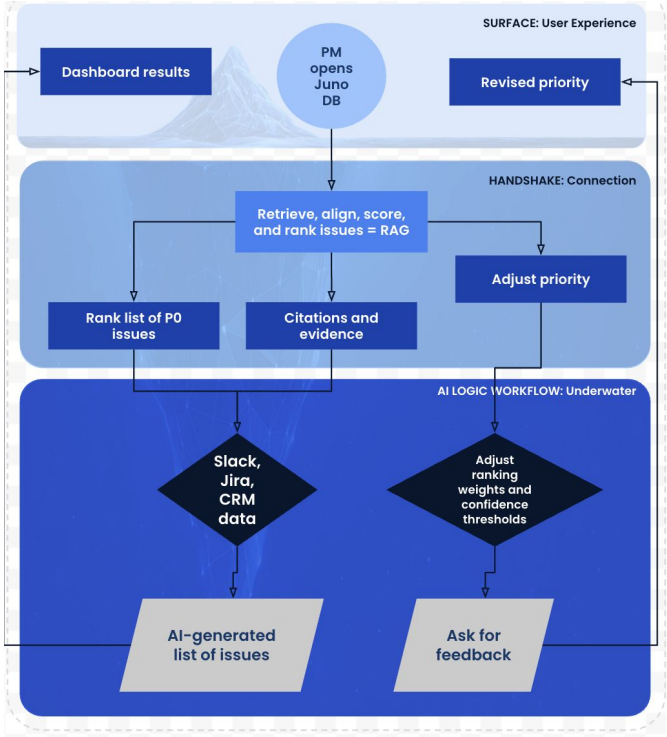
Complete

Live Demo Link: https://prd-whisperer-hub.lovable.app/

Key Screenshots

Signal: **Surface Entry point(s)**	**Trigger**: A PM opens the dashboard, triggering Juno to refresh and rank high-impact issues for the day.
Hidden Logic: **Handshake Major step(s)**	1. **Default Context Initialization**: Set default parameters - timeframe = last 24 hours, scope = all products, goal = prioritize high-impact issues 2. **Signal Retrieval (RAG Layer)**: Pull the most relevant signals from Slack, Jira, Zendesk based on recency, severity, and frequency 3. **Strategic Alignment Check**: Cross-reference signals against RocketShip priorities 4. **Scoring & Ranking Engine**: Assign priority scores (P0–P3) using weighted factors and rank issues
Interaction Maneuver: **Surface Sub-step(s)**	**Display (Dashboard View):** • Ranked list of top issues (P0–P3) • AI-generated summary explaining why these issues are prioritized • Key signals surfaced (e.g., ticket volume, revenue impact, escalation count) **Drill-Down (Evidence Layer):** • Supporting evidence for each issue (tickets, Slack threads, sales notes) • Clear linkage to strategic priorities (e.g., revenue, reliability) **Actions (PM Controls):** • Option to generate a draft PRD or opportunity brief • Ability to adjust priority, provide feedback, or override AI recommendations

Feedback Loop: **Underwater Data Flow**	1. Track PM actions on Juno outputs, including reprioritization, overrides, dismissals, and accepted recommendations 2. Log the before/after change (for example, Juno marked P0, PM changed it to P1) 3. Capture optional PM rationale, such as "low customer impact," "duplicate issue," or "not aligned to current quarter goals" 4. Store these corrections as feedback data tied to the original signals, metadata, and recommendation 5. Use this feedback to refine ranking weights, confidence thresholds, and source weighting over time
Feedback Loop: **Underwater AI-Generated Output**	1. Show when Juno's recommendation was changed by the PM 2. Acknowledge the mismatch, for example: "**Priority adjusted from P0 to P1**" 3. **Ask for feedback on why the change was made →** Surface a short learning prompt, such as: "What did I miss?" with selectable reasons or a text field 4. Use the response to improve future prioritization and reduce repeat errors



Agent Workflow Spec (AWSpec)

Complete

Link to Artifact: *Juno's Agent Workflow Spec (AWSpec)*

Completed Sections

- **Primary Agent Pattern:** **Planner-Executor pattern**

  The system uses a planner agent to orchestrate the workflow and delegate tasks to specialized agents (evidence-gathering, clarification, sizing), then combines their outputs into a final prioritization decision.

  Agent Roles:

  Human (PM): Reviews and validates Juno's prioritization output

  AI (L2 autonomy): Suggests priorities, gathers feedback, and iterates

  Specialized Agents:

  Evidence-Gathering Agent: Collects data from multiple sources

  Clarification Agent: Validates alignment with company strategy (via RAG)

  Sizing Agent: Estimates effort based on historical data

  Planner Agent: Orchestrates flow and sequencing across agents

- **Key Steps:**

  Input: A new ticket or request is received for prioritization

  Evidence Gathering: Agent pulls relevant data from sources

  Clarification: Agent evaluates alignment with company goals using RAG

Evaluation Plan

Complete

Definition of "Good"

- ➔ *Strategic Alignment : Output strictly follows the RocketShip Strategy (e.g., correct prioritization like P0/P1 when required).*
- ➔ *No Hallucinations : All requirements, tickets, and insights are grounded in the provided transcripts (no invented features or domains).*
- ➔ *Clear & Structured PRD Output : Produces well-formed PRDs with correct priorities, complete sections, and actionable details (no missing fields like ticket priority or risks).*

Eval Methodology

1. Active Feedback
   - Accept / Reject Juno's prioritization output -> (Thumbs / rating on whether priorities are correct)
2. Passive Behavior Tracking (from your blue input)
   - Log if user re-asks for priorities (P0–P3)
   - Track if user manually adjusts priorities before adding to roadmap (P0/P1)
3. Human Evaluation Rubric
   - Strategic Alignment : Does Juno correctly prioritize P0/P1 when explicitly requested?
     Failure signal: returns a P3 when P0/P1 requested
   - Hallucination Check: Did Juno introduce tickets outside the request?
     Failure signal: ticket from unrelated domain
   - Technical/Risk Depth (1–5): How effectively does Juno identify hidden risks, edge cases, or technical debt not explicitly mentioned in the input data? Examples: cascading system failures, Data inconsistency risks, Cross-team dependency bottlenecks. On a scale of 4. Reasoning - As a Senior PM level, as opposed to a standard associate, a generic response or PM level details.

Risk Mitigation

Group Insights: Building Process

Complete

Friction Points

- *The AWSpec, actors, and pattern plan we were challenged with selecting the right architecture for multi-agent flows.*
- *In the AI strategy ine pager what is In and Out of scope , the balance between solving the problem accurately and having a strong business value.*

Key Learnings

- *The evaluation techniques and how it's a mix of rules to avoid failures and yet it takes HITL to make a call on the quality and build trust*
- *RAG ensures grounded accuracy, while Agents enable multi-step reasoning and action and the real power comes from combining both.*

Aha! Moment

- *That there are more memory models than we were listed in the modules*
- *My company's copilot is just a mix of RAGs and couple on well prompted agents.*